ICT ASSIGNMENT NUMBER 1

NAME:
Muhammad Saad Hayat

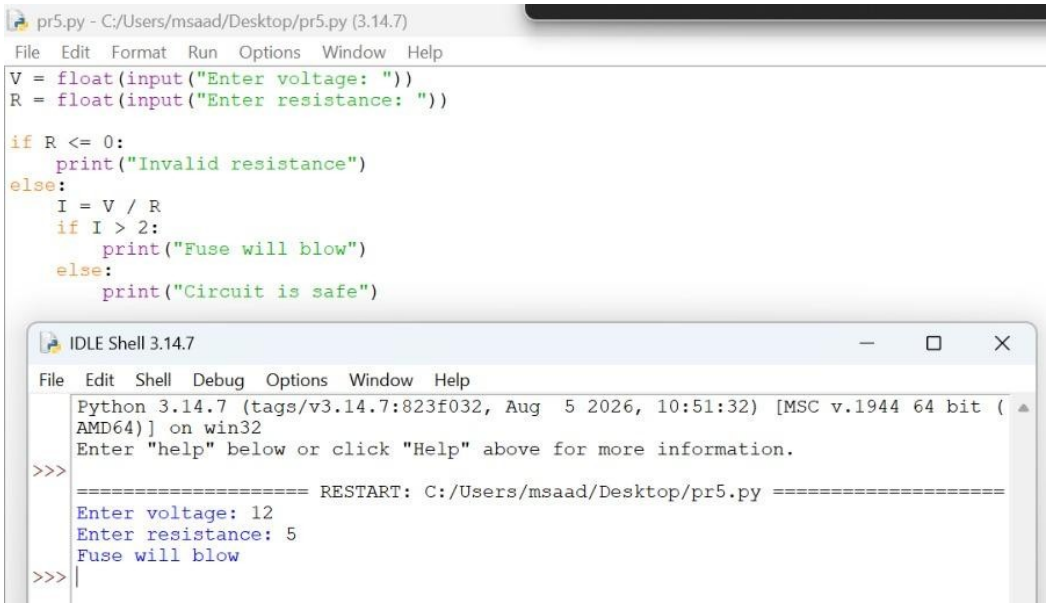
DEGREE:
48EEB

INSTRUCTOR:
Sir Shahzad

Part A: Conditionals

A1

Ohm's law check. Take voltage V (volts) and resistance R (ohms) from the user and calculate current I = V/R. If I is more than 2 A, print "fuse will blow", otherwise print "circuit is safe". If R is 0 or negative, print "invalid resistance" instead.

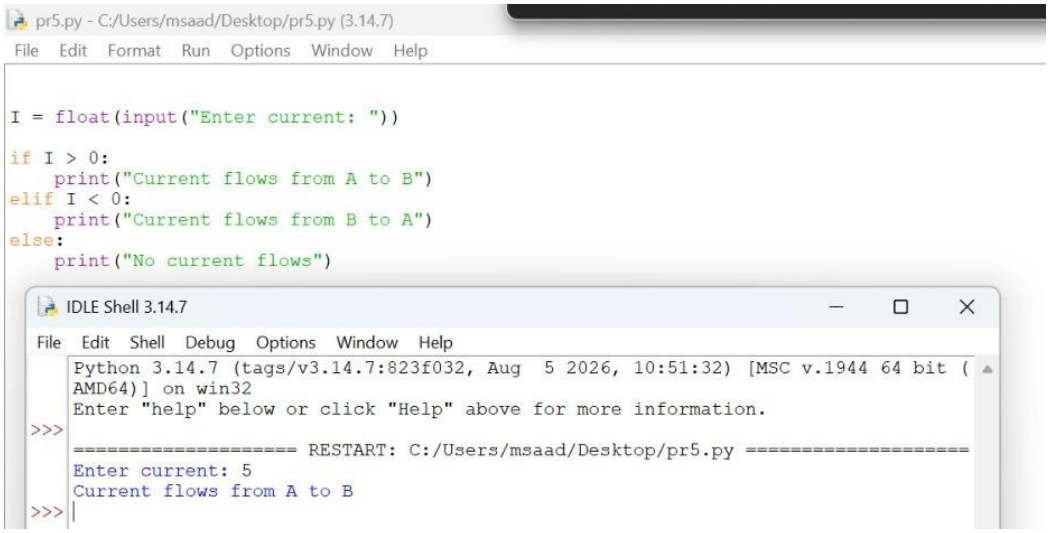


Working

The program reads V and R as numbers. If R is 0 or negative, it prints "Invalid resistance". Otherwise it calculates I = V / R and checks if I is greater than 2 A. For 12 V and 5 Ω, I = 2.4 A, so the output is "Fuse will blow".

A2

Current direction. Take a current value from the user. If it is positive, print "current flows from A to B". If it is negative, print "current flows from B to A". If it is 0, print "no current flows".

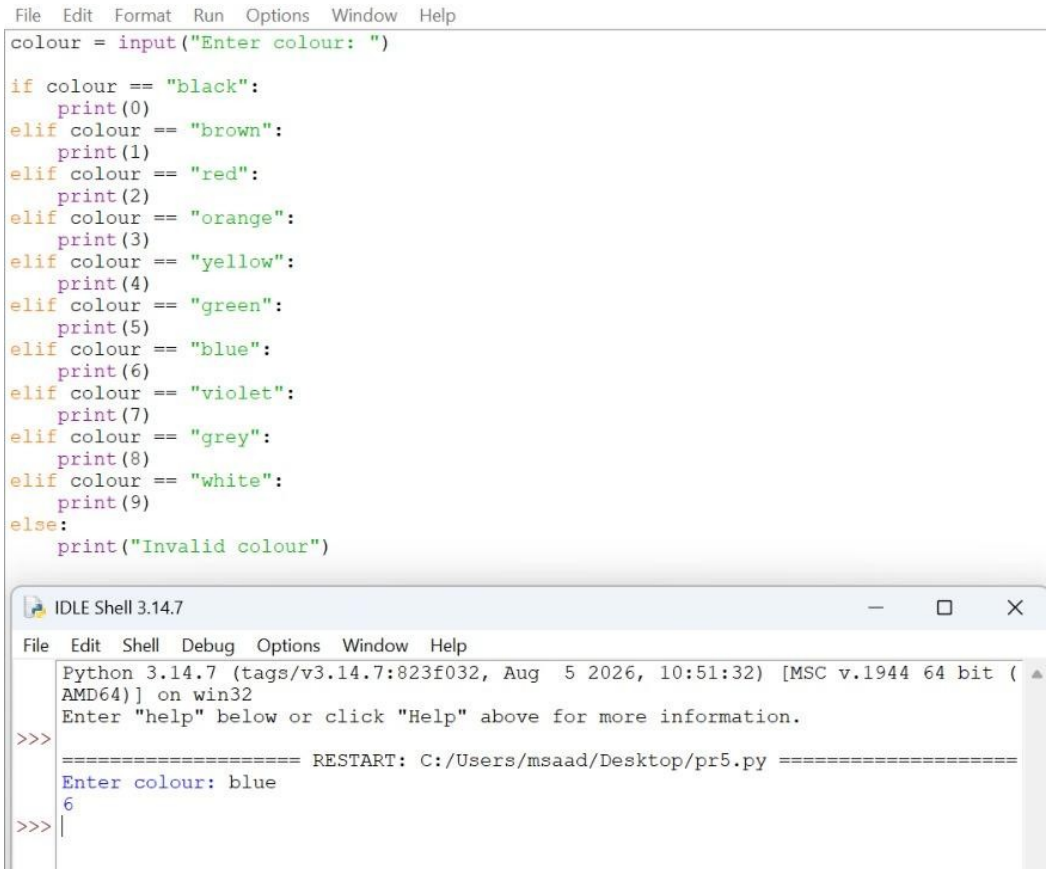


Working

The program reads the current I and checks its sign using if, elif and else. A positive value prints that current flows from A to B, a negative value prints B to A, and zero prints that no current flows. The input 5 is positive, so the output is "Current flows from A to B".

A3

Resistor color code. Take the color of the first band of a resistor and print its digit: black 0, brown 1, red 2, orange 3, yellow 4, green 5, blue 6, violet 7, gray 8, white 9. For any other input, print "invalid color".

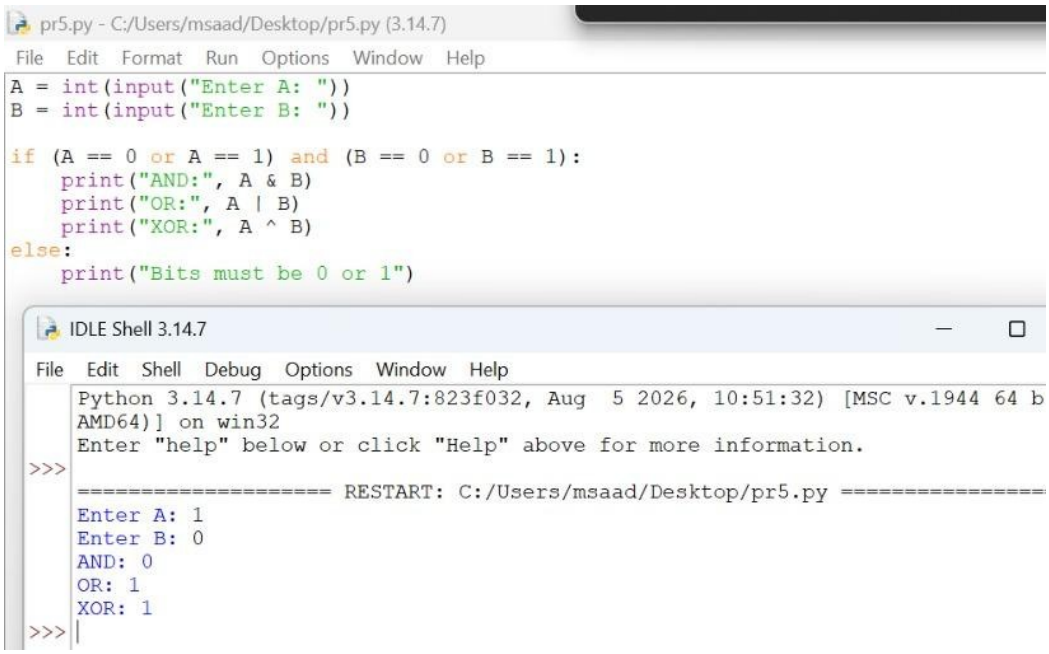


Working

The program reads the colour name and compares it with each colour using an if-elif chain. When a colour matches, the program prints its digit, and if nothing matches, the else part prints "Invalid colour". The input "blue" matches the blue condition, so the output is 6.

A4

Logic gates. Take two bits A and B (each 0 or 1) from the user and print the output of the AND, OR and XOR gates. If either input is not 0 or 1, print "bits must be 0 or 1".

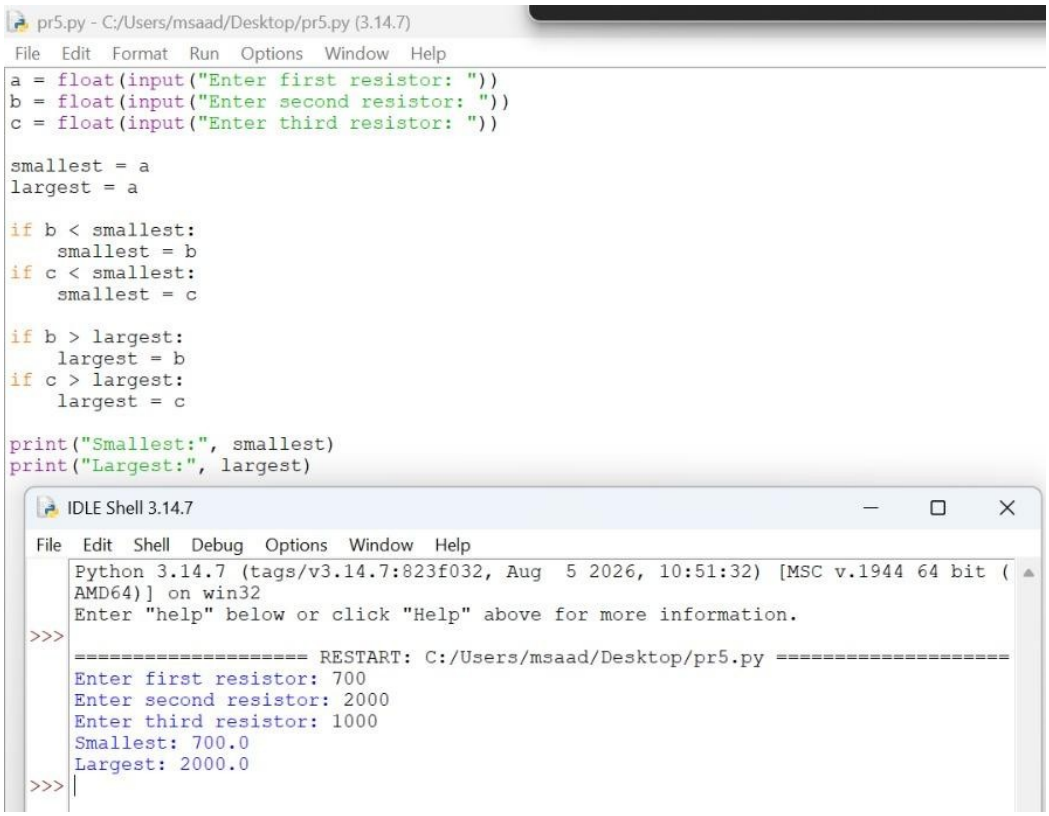


Working

The program reads A and B as integers and first checks that both are 0 or 1. If they are, it prints A & B for AND, A | B for OR and A ^ B for XOR; otherwise it prints "Bits must be 0 or 1". For A = 1 and B = 0, the output is AND 0, OR 1 and XOR 1.

A5

Smallest and largest resistor. Take three resistor values from the user and display the smallest and the largest of them. Use only if, elif and else.



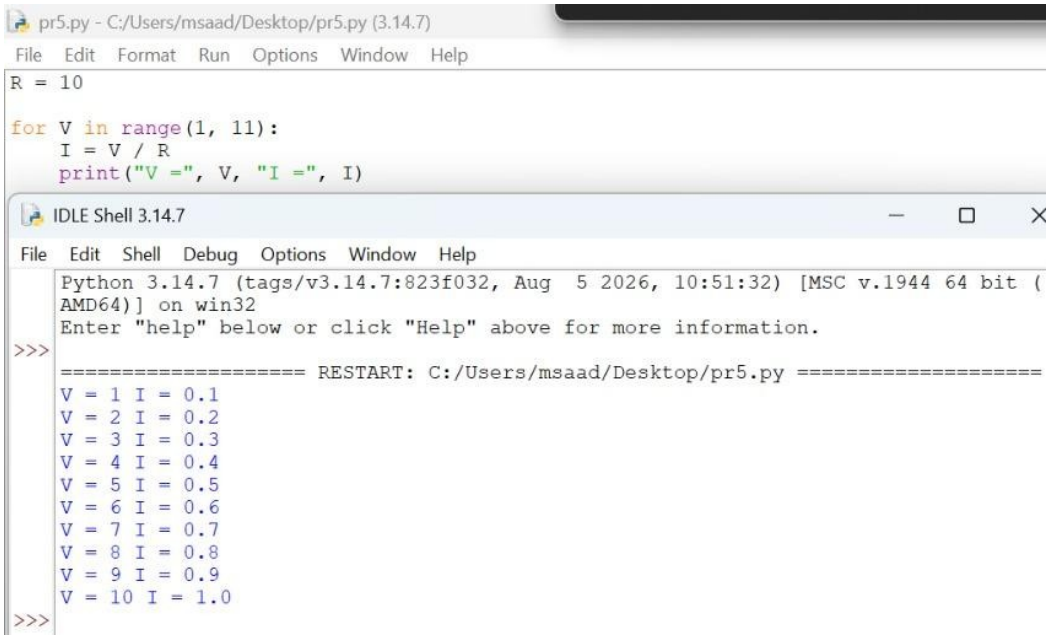
Working

The program reads three resistor values and assumes the first one is both the smallest and the largest. It then compares the second and third values with these and updates smallest or largest when a better value is found. For 700, 2000 and 1000, the output is smallest 700.0 and largest 2000.0.

Part B: Loops

B1

Ohm's law table. A resistor has a fixed value of 10 Ω. Using a for loop, print the current through it for supply voltages from 1 V to 10 V in steps of 1 V.

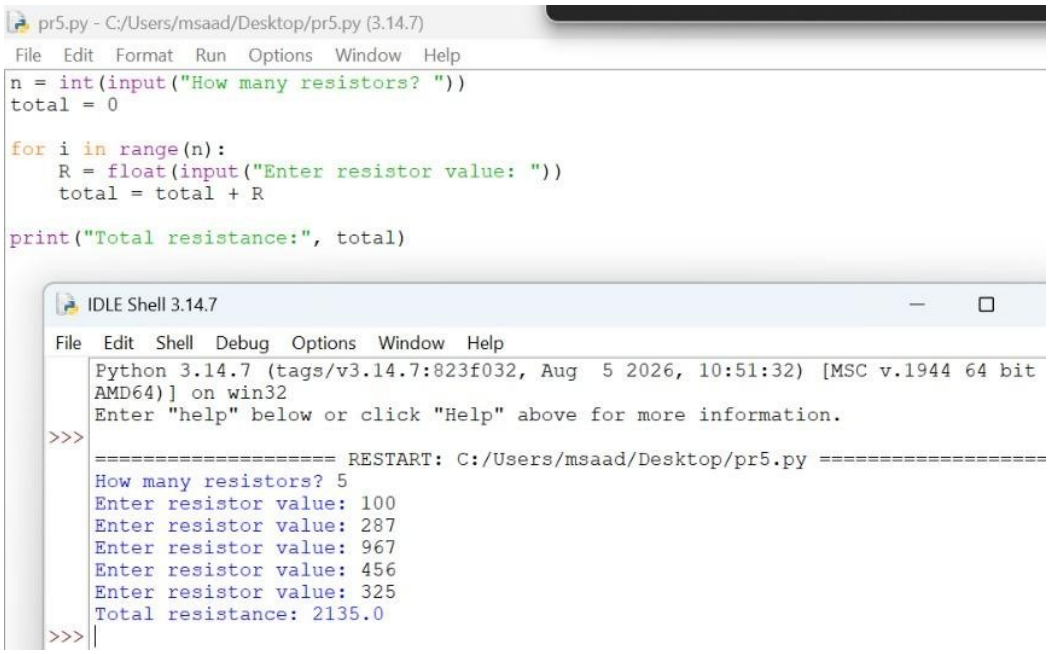


Working

The resistor R is fixed at 10 Ω. The for loop runs V from 1 to 10 using range(1, 11), and each time it calculates I = V / R and prints both values. The current increases by 0.1 A for every 1 V, from 0.1 A to 1.0 A.

B2

Resistors in series. Ask the user how many resistors are connected in series, then use a loop to read the value of each resistor and print the total series resistance.

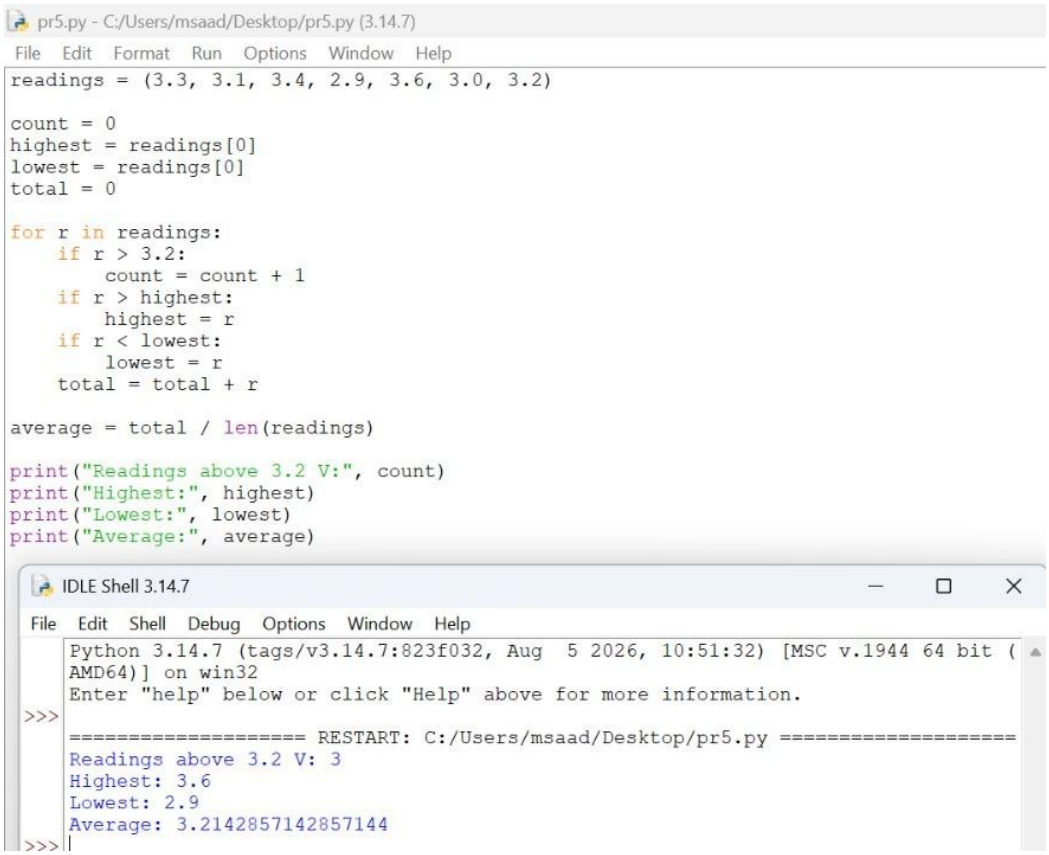


Working

The program first asks how many resistors there are. The for loop then runs that many times, reads each resistor value and adds it to total, because series resistance is the sum of all resistances. For the five values entered, the total resistance is 2135.0 Ω.

B3

Sensor readings. A sensor gives these voltage readings: 3.3, 3.1, 3.4, 2.9, 3.6, 3.0, 3.2. Using one loop, find and print how many readings are above 3.2 V, the highest reading, the lowest reading, and the average reading.

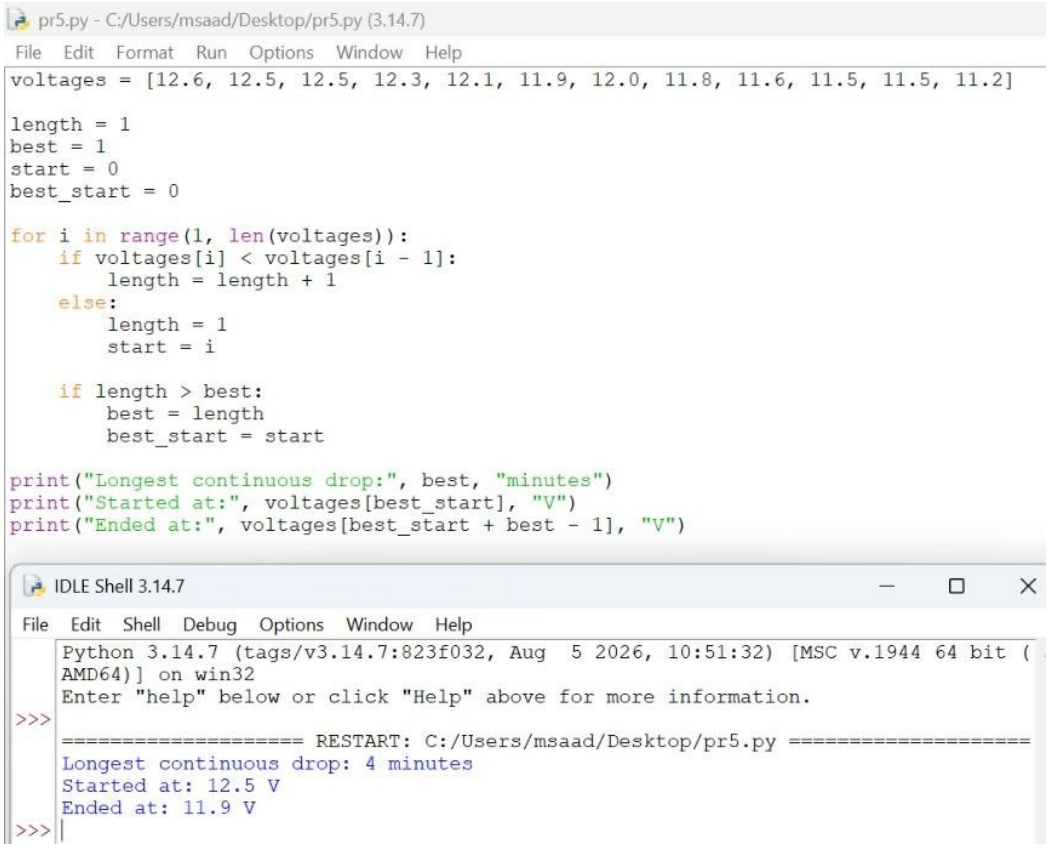


Working

A single for loop goes through every reading. Inside the loop it counts the readings above 3.2, updates the highest and lowest values, and adds each reading to total. After the loop, the average is total divided by the number of readings. The output is 3 readings above 3.2 V, highest 3.6, lowest 2.9 and average 3.214.

B4

Continuous discharge analysis. A battery management system records voltage readings every minute. Given the hard-coded list of voltages 12.6, 12.5, 12.5, 12.3, 12.1, 11.9, 12.0, 11.8, 11.6, 11.5, 11.2, use a loop to find the longest consecutive streak of strictly decreasing voltage readings. Do not use built-in sorting and grouping functions. Print the length of the longest streak as well as the starting and ending voltages of that specific sequence.

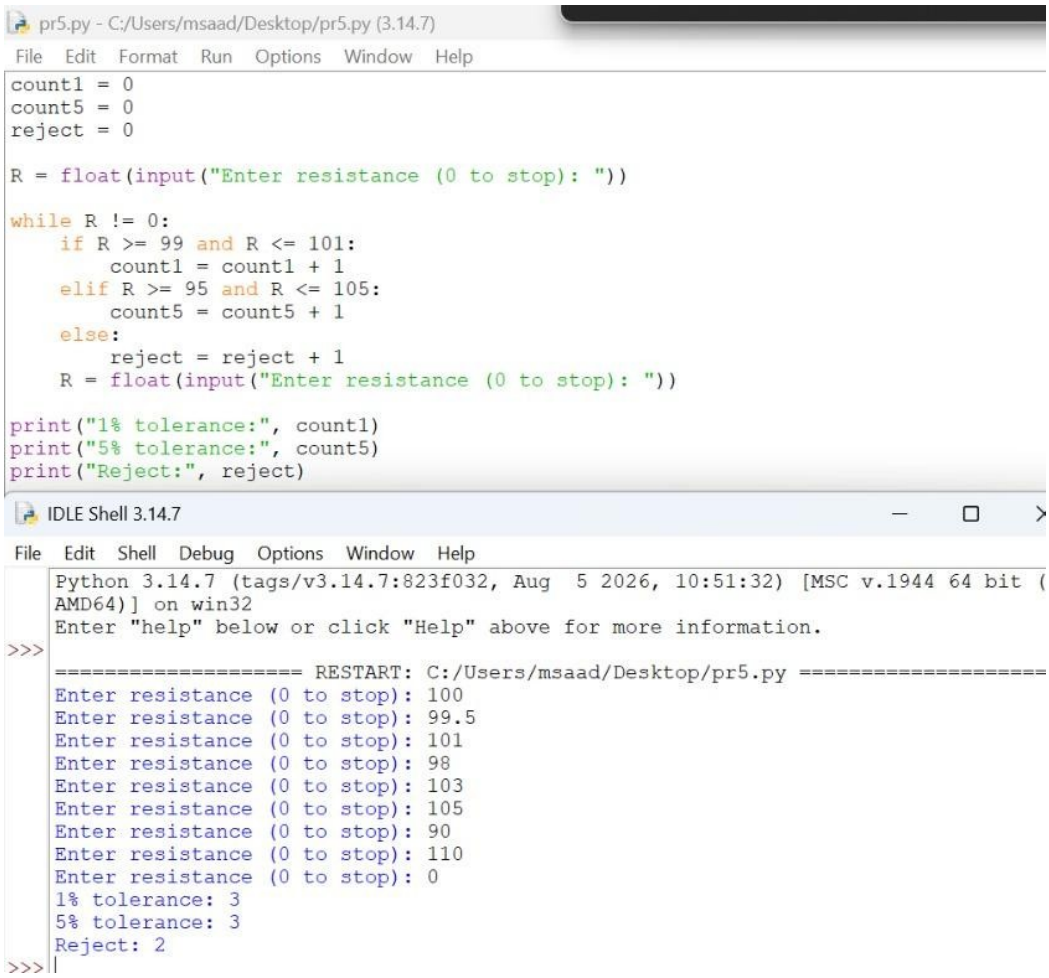


Working

The loop compares each reading with the one before it. If the reading is smaller, the current streak length increases by one; otherwise the streak restarts from the current position. Whenever the current streak is longer than the best one, the best length and its starting position are saved. The longest streak is 4 minutes, from 12.5 V to 11.9 V.

B5

Resistor tolerance sorting. You are testing a batch of manufactured 100 Ω resistors to bin them by tolerance. Write a program with a while loop that continuously asks the user to input a measured resistance value. The loop should stop when the user enters 0. Sort the inputs into three running counts: 1% tolerance (99 to 101 Ω), 5% tolerance (95 to 105 Ω), and reject (outside 95 to 105 Ω). Print the final count of resistors in each bin once the loop terminates.



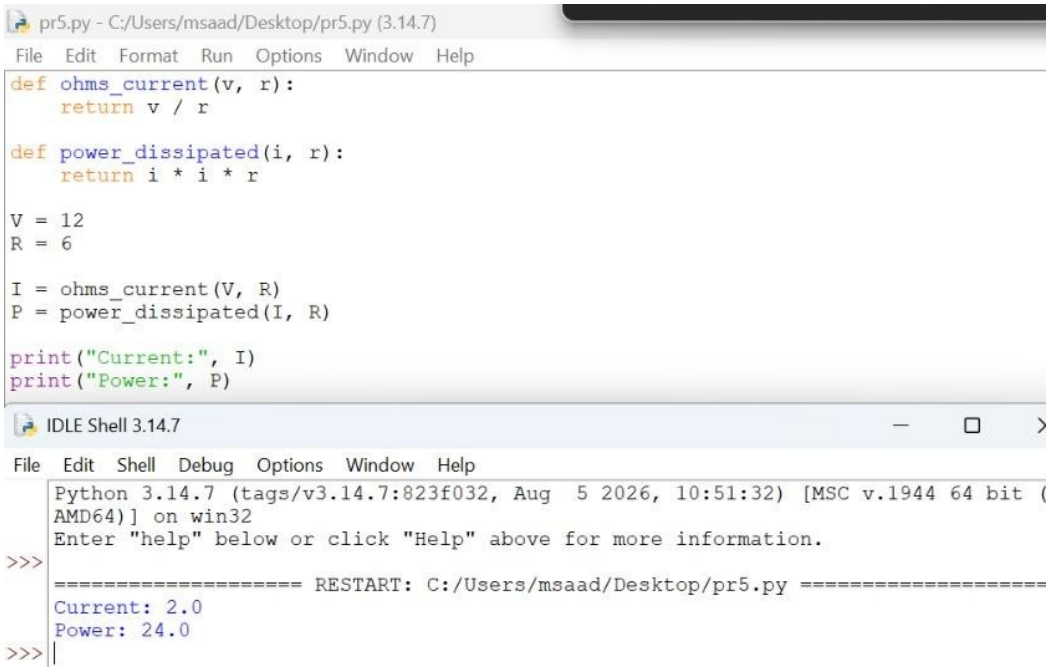
Working

The while loop keeps running until the user enters 0. For each value, the if-elif-else block adds one to the 1% count if the value is from 99 to 101, to the 5% count if it is from 95 to 105, and to the reject count otherwise. When 0 is entered, the loop ends and the three counts are printed: 3, 3 and 2.

Part C: Functions

C1

Current and power. Write two functions: ohms_current that returns the current I = V/R, and power_dissipated that returns the power P = I²R. Test them for V = 12 V and R = 6 Ω.

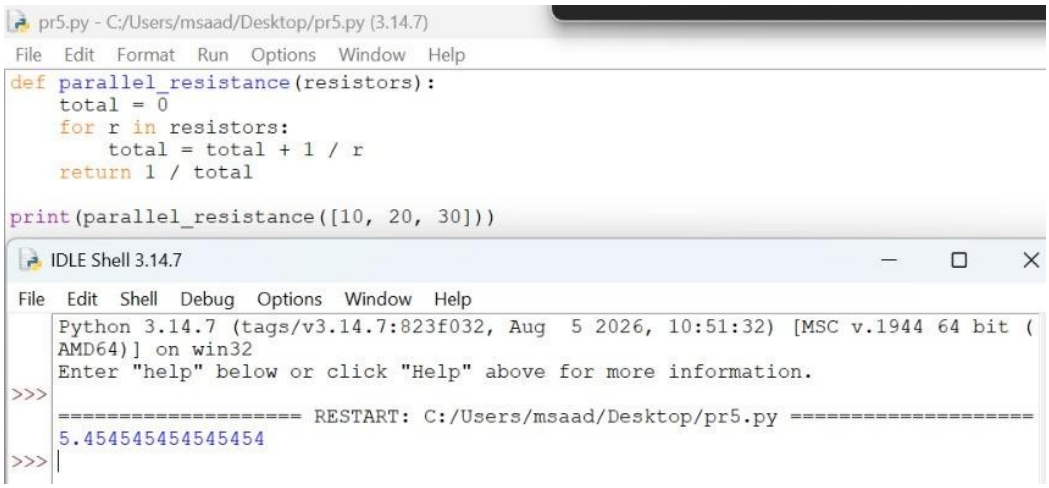


Working

The function ohms_current returns v / r, and power_dissipated returns i * i * r. The program first finds the current for 12 V and 6 Ω, then passes that current to the power function. The output is a current of 2.0 A and a power of 24.0 W.

C2

Parallel resistance. Write a function parallel_resistance that takes a list of resistor values and returns the equivalent resistance of the parallel combination.

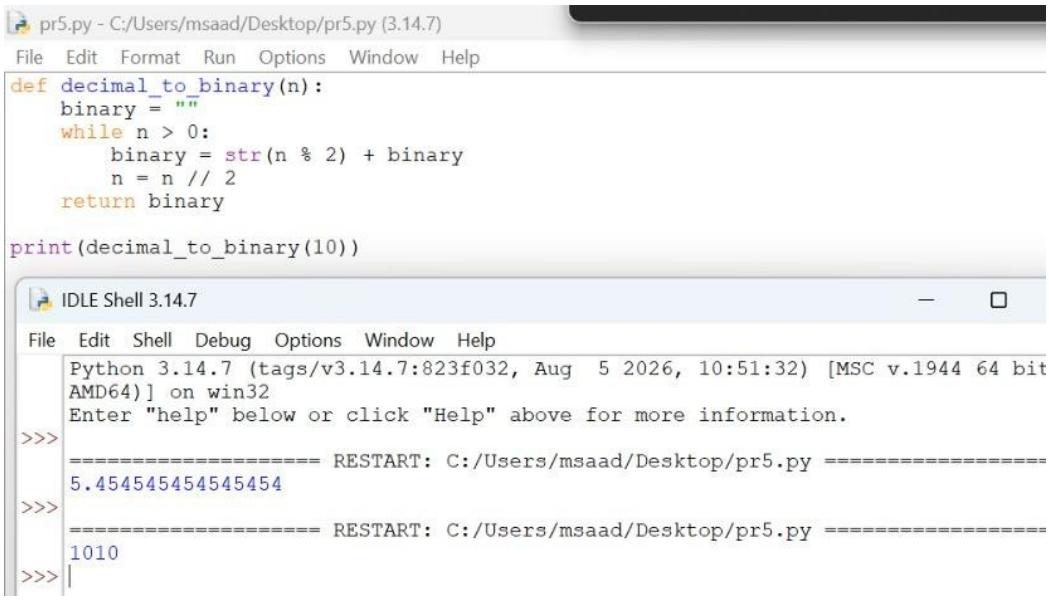


Working

The function goes through the list and adds 1/r for every resistor to total. Since the parallel formula is 1/Req = 1/R1 + 1/R2 + ..., it returns 1 / total at the end. For 10, 20 and 30 Ω, the equivalent resistance is about 5.45 Ω.

C3

Decimal to binary. Write a function decimal_to_binary that converts a positive decimal number to a binary string using repeated division by two.

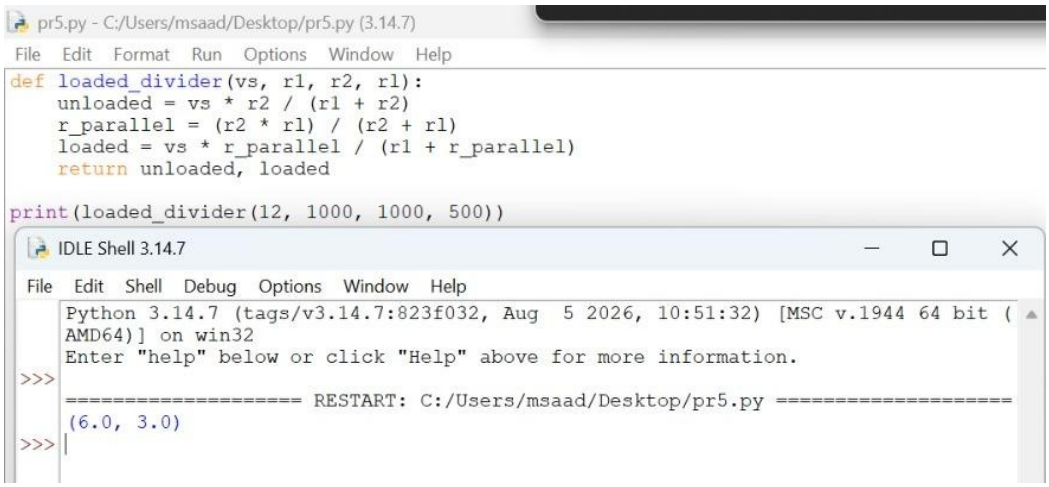


Working

The function starts with an empty string and repeats while n is greater than 0. Each time, the remainder n % 2 is added to the front of the string and n is divided by 2 using n // 2. For the number 10, the remainders give the binary string 1010.

C4

Loaded voltage divider. The standard voltage divider formula assumes no current is drawn by the load, which is rarely true in a practical circuit. Write a function loaded_divider(vs, r1, r2, rl) that calculates the true output voltage when a load resistor RL is attached in parallel with R2. The function must first compute the parallel equivalent of R2 and RL, then apply the voltage divider formula with R1. Return both the unloaded (theoretical) voltage and the actual loaded voltage.

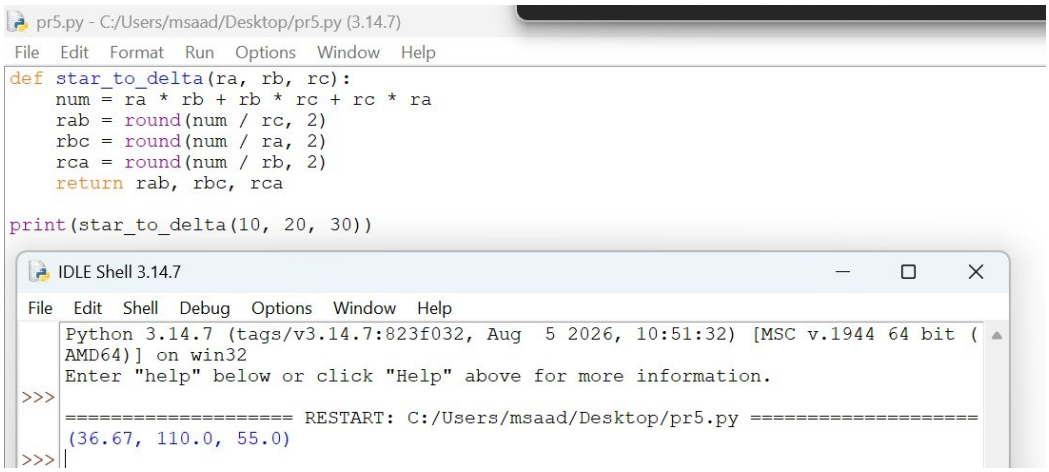


Working

The function first finds the unloaded output voltage using the normal divider formula. It then finds the parallel resistance of r2 and rl and uses it in the divider formula with r1 to get the loaded voltage. For 12 V, R1 = R2 = 1000 Ω and RL = 500 Ω, the output voltage drops from 6.0 V to 3.0 V.

C5

Star-Delta (Y-Δ) Transformation. Write a function star_to_delta(ra, rb, rc) that takes three resistance values from a Star (Y) network and converts them to the equivalent Delta (Δ) network. The function must calculate the numerator factor first to avoid redundant calculations, using the formula: R_numerator = Ra·Rb + Rb·Rc + Rc·Ra. Then calculate the three Delta resistors: Rab = R_numerator / Rc, Rbc = R_numerator / Ra, and Rca = R_numerator / Rb. Return the three newly calculated resistance values rounded to two decimal places. Sample run: star_to_delta(10, 20, 30) -> (36.67, 110.0, 55.0)



Working

The function first calculates the numerator once as ra*rb + rb*rc + rc*ra, which avoids repeating the same calculation. It then divides this numerator by rc, ra and rb to get rab, rbc and rca, rounds each to two decimal places and returns them. For 10, 20 and 30 Ω, the numerator is 1100 and the result is (36.67, 110.0, 55.0).

GitHub Link: https://github.com/msaadhayat555/test.git